

Algoritmy a grafy 1 (BI-AG1), Přednáška č. 11

Minimální kostry grafu

Dušan Knop, Michal Opler, Tomáš Valla

Fakulta informačních technologií
České vysoké učení technické v Praze
<https://courses.fit.cvut.cz/BI-AG1/>

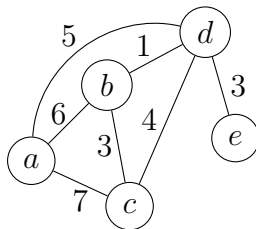


ZS 2025/2026, 8. prosince 2025

(Verze dokumentu: 14. 12. 2025 19:21)

Problém minimální kostry

- Napadl sníh a všechny ulice města jsou zasněžené.
- Které ulice prohrneme, aby šlo dojet odkudkoliv kamkoliv, a přitom nám prohrnutí sněhu dalo co nejmeně práce?



- Tato otázka vede na hledání tzv. **minimální kostry grafu**.

Problém minimální kostry

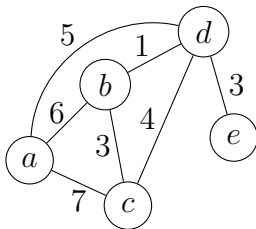
Popišme nyní problém formálně.

Připomenutí: Kostra grafu G je podgraf, který obsahuje všechny vrcholy a je to strom, a má tedy $|V(G)| - 1$ hran.

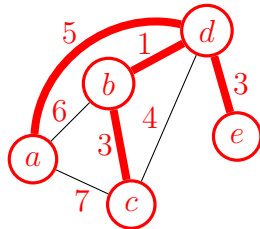
Definice

- Mějme souvislý neorientovaný graf $G = (V, E)$. Každé hraně $e \in E$ přiřadíme číselnou **váhu** $w(e)$, kde $w: E \rightarrow \mathbb{R}$. Takový graf se nazývá **hranově ohodnocený**.
- Váhovou funkci můžeme přirozeně rozšířit na podgrafy: **Váha** $w(H)$ podgrafu $H \subseteq G$ je součet vah jeho hran.
- Kostra hranově ohodnoceného grafu G je **minimální**, pokud má mezi všemi jeho kostrami nejmenší váhu.

Problém minimální kostry



(a) Graf ohodnocený váhovou funkcí



(b) Minimální kostra grafu

Jarníkův algoritmus

- Nejjednodušší algoritmus pro hledání minimální kostry pochází z roku 1930, kdy ho vymyslel český matematik Vojtěch Jarník.
- Tehdy myšlenka zapadla a až později byla několikrát znovuoobjevena – proto se algoritmu říká též Primův.

Myšlenka Jarníkova algoritmu:

- Začneme se stromem, který obsahuje libovolný jeden vrchol a žádné hrany.
- Vybereme nejlehčí hranu incidentní s tímto vrcholem.
- Přidáme ji do stromu i s novým koncovým vrcholem.
- Postup opakujeme: v každém dalším kroku přidáváme nejlehčí z hran, které vedou mezi vrcholy dosud vytvořeného stromu a zbytkem grafu.
- Takto pokračujeme, dokud nevznikne celá kostra.

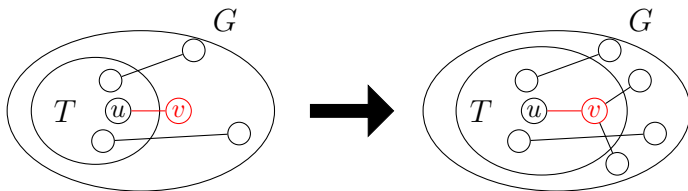
Jarníkův algoritmus

Vstup: Souvislý hranově ohodnocený graf G

Výstup: Minimální kostra G

Algoritmus **MinKostrajarník** ($G = (V, E), w: E \rightarrow \mathbb{R}$)

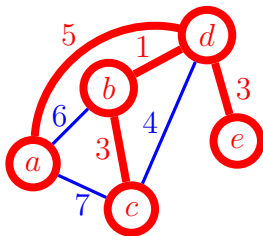
- (1) $v_0 :=$ libovolný vrchol grafu
- (2) $T :=$ strom obsahující vrchol v_0 a žádné hrany
- (3) Dokud existuje hrana $\{u, v\}$ taková,
že $u \in V(T)$ a $v \notin V(T)$:
- (4) Přidej nejlehčí takovou hranu spolu s v do T
- (5) Vrať T



Jarníkův algoritmus – příklad

Algoritmus **MinKostrajarník** ($G = (V, E), w: E \rightarrow \mathbb{R}$)

- (1) v_0 := libovolný vrchol grafu
- (2) T := strom obsahující vrchol v_0 a žádné hrany
- (3) Dokud existuje hrana $\{u, v\}$ taková,
že $u \in V(T)$ a $v \notin V(T)$:
- (4) Přidej nejlehčí takovou hranu spolu s v do T
- (5) Vrať T



Hladové algoritmy

- Jarníkův algoritmus je typickým příkladem tzv. **hladového algoritmu** (angl. *greedy algorithm*).
- V každém okamžiku vybíráme **lokálně** nejlepší hranu.
- Minimální kostra je jedním z řídkých případů, kdy hladový algoritmus skutečně spočte optimální řešení.
- Dokážeme postupně, že Jarníkův algoritmus skutečně vrátí minimální kostru.

Důkaz korektnosti Jarníkova algoritmu

- Nejprve ukážeme, že se Jarníkův algoritmus spuštěný na souvislém hranově ohodnoceném grafu zastaví a vygeneruje kostru.
- Dále budeme pro jednoduchost předpokládat hranově ohodnocený graf s unikátními vahami a dokážeme, že má jedinou minimální kostru a Jarníkův algoritmus ji vytvoří.
- K tomu zavedeme pojem elementárního řezu grafu.
- Poté ukážeme, že Jarníkův algoritmus vydá minimální kostru i v případě, že váhy nejsou unikátní (viz graf v předchozím příkladu).

Konečnost Jarníkova algoritmu

Lemma (o konečnosti Jarníkova algoritmu)

Jarníkův algoritmus se po $|V| - 1$ iteracích zastaví a vydá nějakou kostru zadaného grafu $G = (V, E)$.

Důkaz.

- Graf T konstruovaný algoritmem vzniká z jednoho vrcholu postupným přidáváním listů.
- Takže T je v každém okamžiku výpočtu strom.
- V každé iteraci se $|V(T)|$ zvětší o 1.
- Dokud $V(T) \neq V(G)$, musí díky souvislosti G existovat hrana mezi $V(T)$ a $V(G) \setminus V(T)$, kterou může algoritmus přidat do T .
- To znamená, že algoritmus se zastaví, až když $V(T) = V(G)$.
- Tehdy je ale T kostra. □

Elementární řezy v grafu

Definice

Nechť A je nějaká podmnožina vrcholů grafu $G = (V, E)$ a B její doplněk, tj. $B = V \setminus A$.

Množině všech hran, které leží jedním vrcholem v A a druhým v B , budeme říkat **elementární řez** grafu G určený množinami A a B .

Lemma (o řezech v grafu s unikátními vahami)

Nechť G je souvislý ohodnocený graf s unikátními vahami, R nějaký jeho elementární řez a e nejlehčí hrana tohoto řezu.

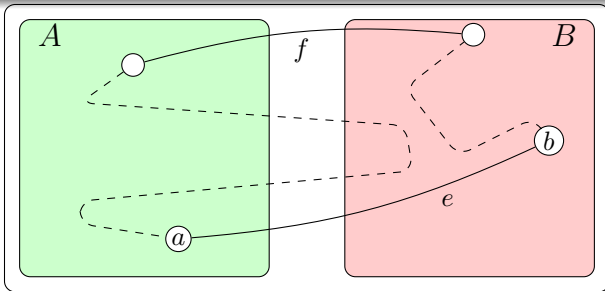
Pak (každá) minimální kostra grafu G obsahuje hranu e .

Důkaz.

- Dokážeme obměněnou implikaci: Pokud nějaká kostra T neobsahuje hranu e , pak není minimální.
- ...

Pokračování důkazu.

- Označme A a B množiny vrcholů určující elementární řez R .
- Hrana $e = \{a, b\}$, $a \in A$, $b \in B$, je nejlehčí v R .
- Nechť T je kostra neobsahující e .
- Protože T je strom, existuje v něm jediná cesta $P(a, b)$ a ta musí alespoň jednou překročit elementární řez R .
- Nechť $f = \{a', b'\}$, $a' \in A$, $b' \in B$, je libovolná hrana, kde se to stalo.
- ...



Dokončení důkazu.

- Nyní z kostry T odebereme hranu f .
- Tím se kostra rozpadne na dva stromy T_a, T_b , z nichž jeden obsahuje a a druhý b .
- Přidáním hrany e stromy opět propojíme a tím získáme jinou kostru T' .
- Platí $w(T') = w(T) - w(f) + w(e)$.
- Protože e je nejlehčí hrana v elementárním řezu a váhy jsou unikátní, musí platit $w(f) > w(e)$.
- Proto $w(T') < w(T)$ a tudíž T není minimální kostra. □

Správnost Jarníkova algoritmu

Věta (o minimální kostře)

Souvislý graf s unikátními vahami má právě jednu minimální kostru a Jarníkův algoritmus tuto kostru vytvoří.

Důkaz.

- Jarníkův algoritmus v každém kroku vybere jedinečnou nejlehčí hranu elementárního řezu mezi vrcholy dosud vytvořeného stromu a zbytkem grafu, která je vždy dle lemmatu obsažena v každé minimální kostře.
- To ale znamená, že kostra vytvořená Jarníkovým algoritmem je podgrafem každé minimální kostry.
- Protože ale všechny kostry daného grafu mají stejný počet hran, znamená to, že vytvořená kostra je všem minimálním kostrám rovna a je tudíž unikátní. □

Jednoznačnost minimální kostry

Důsledek

Minimální kostra je jednoznačně určena uspořádáním hran podle vah, na konkrétních hodnotách vah nezáleží.

Důkaz.

Jarníkův algoritmus váhy hran mezi sebou pouze porovnává. ☐

- Pokud tedy pro každý elementární řez grafu G existuje právě jedna nejlehčí hrana, pak má G právě jednu minimální kostru.
- Protipříklad na Slajdu 7 ale ukazuje, že neplatí obrácená implikace, tedy: Pokud má graf G právě jednu minimální kostru, pak pro každý elementární řez grafu G existuje právě jedna nejlehčí hrana.

Korektnost Jarníkova alg. pro neunikátní váhy

- Pokud nejsou váhy unikátní, může v dané iteraci existovat několik nejlehčích hran $\{a, b\}$, $a \in V(T)$, $b \notin V(T)$ a Jarníkův algoritmus vždy jednu z nich vybere.
- Lemma o řezech stačí upravit následovně.

Lemma (o řezech v grafu s opakujícími se vahami hran)

Nechť G je souvislý ohodnocený graf, R je elementární řez v G a e je libovolná nejlehčí hrana v R . Pro každou minimální kostru T' existuje minimální kostra T taková, že

- * T' a T se mohou lišit pouze na hranách obsažených v R a
- * T obsahuje e .

Důsledek

Pokud G obsahuje více minimálních koster díky existenci hran se stejnou vahou, pak Jarníkův algoritmus jednu z nich zkonstruuje.

Lemma (o řezech v grafu s opakujícími se vahami hran)

Nechť G je souvislý ohodnocený graf, R je elementární řez v G a e je libovolná nejlehčí hrana v R . Pro každou minimální kostru T' existuje minimální kostra T taková, že

- * T' a T se mohou lišit pouze na hranách obsažených v R a
- * T obsahuje e .

Důkaz.

- Označme A a B množiny vrcholů, kterými je určen elementární řez R .
- Nechť T' neobsahuje e a nechť $e = \{a, b\}$.
- Buď P unikátní cesta mezi vrcholy a a b v T' .
- Cesta P prochází elementárním řezem R .
- Nechť f je libovolná hrana v $E(P) \cap R$.
- Potom $T := T' - f + e$ je kostra a platí, že $w(T) \leq w(T')$. $\square$

Složitost Jarníkova algoritmu

Pozorování

Naivní implementace Jarníkova algoritmu nad grafem $G = (V, E)$, reprezentovaným seznamem sousedů má časovou složitost $O(|V| \cdot |E|)$ ($(|V| - 1)$ -krát hledáme nejlehčí hranu v seznamu nejvýše $|E|$ hran) a paměťovou složitost $O(|V| + |E|)$.

- V Jarníkově algoritmu opakovaně vybíráme nejlehčí hranu a odstraňujeme ji ze seznamu.
- Z Přednášky 4 víme, že je efektivní odebrání minim z dynamicky se měnící množiny dokáže binární halda.
- Ukažme si nyní tedy implementaci pomocí binární haldy.

Jarníkův algoritmus pomocí minimové binární haldy

- Vrcholy grafu G jsou udržovány v binární haldě H uspořádané podle hodnoty klíčů d , určujících „vzdálenost“ vrcholů od dosud vybudovaného stromu v daném kroku algoritmu.
- Na počátku je $d(v)$ nekonečno pro všechny vrcholy a zvolený v_0 má nulu.
- Z haldy vybereme pomocí **ExtractMin** vrchol u s minimálním d , čímž ho přidáme ke stromu a přepočteme pro jeho sousedy v vzdálenosti d .
- Pokud se jejich d změnilo, pomocí **DecreaseKey** aktualizujeme haldu a vrchol u se stává jejich předchůdcem.
- Minimum aktualizované haldy je rovno váze nejlehčí hrany v elementárním řezu mezi novým T a zbytkem grafu.

Složitost Jarníkova algoritmu s binární haldou

Věta (o časové složitosti Jarníkova algoritmu s binární haldou)

Časová složitost `MinKostrajarník` při použití binární haldy je $O(|E| \log |V|)$.

Důkaz.

- Inicializace zabere $O(|V|)$.
- Operace s binární haldou:
 - ▶ Provedeme $|V|$ -krát operaci `HeapExtractMin` – celkem $O(|V| \log |V|)$.
 - ▶ Každá hrana může způsobit volání operace `HeapDecreaseKey` – celkem $O(|E| \log |V|)$.

Dohromady dostáváme

$$O(|V|) + O(|V| \log |V|) + O(|E| \log |V|) = O(|E| \log |V|).$$



Kruskalův algoritmus

- Kruskalův algoritmus je také založen na hladovém přístupu: začne lesem tvořeným pouze samotnými vrcholy bez hran a zkouší přidávat hrany od nejlehčí po nejtěžší a zahazuje ty, které by vytvořily cyklus.

Vstup: Souvislý hranově ohodnocený graf $G = (V, E)$.

Nechť $n = |V|$ a $m = |E|$.

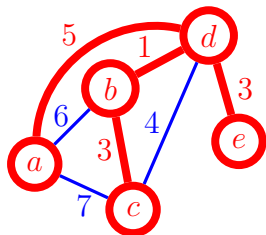
Algoritmus **MinKostraKruskal**($G = (V, E), w: E \rightarrow \mathbb{R}$)

- (1) Seřaď hrany podle vah: $w(e_1) \leq \dots \leq w(e_m)$
- (2) $T := (V, \emptyset)$ //počáteční les bez hran
- (3) Pro $i := 1, \dots, m$ opakuj:
- (4) označ u, v krajní vrcholy hrany e_i
- (5) Pokud u a v leží v různých komponentách lesa T :
- (6) $E(T) := E(T) \cup \{e_i\}$
- (7) Vrať T

Kruskalův algoritmus – příklad

Algoritmus **MinKostrKruskal** ($G = (V, E), w: E \rightarrow \mathbb{R}$)

- (1) Uspořádej hrany podle vah: $w(e_1) \leq \dots \leq w(e_m)$
- (2) $T := (V, \emptyset)$
- (3) Pro $i := 1, \dots, m$ opakuj:
 - (4) označ u, v krajní vrcholy hrany e_i
 - (5) Pokud u a v leží v různých komponentách lesa T :
 - (6) $E(T) := E(T) \cup \{e_i\}$
 - (7) Vrať T



bd	de	bc	cd	ad	ab	ac
1	3	3	4	5	6	7
i	i	i	i	i	i	i

Správnost Kruskalova algoritmu

Lemma

Kruskalův algoritmus se zastaví a vydá minimální kostru.

Důkaz.

- Konečnost: cyklus se vykoná m -krát.
- Správnost: Ukážeme nejdříve, že pokud algoritmus přidá hranu $e = \{u, v\}$ do T , pak e leží v minimální kostře.
- Pokud algoritmus e přidá, stane se tak v okamžiku, kdy se vrcholy u a v nacházejí v nějakých dvou rozdílných stromech T_u a T_v lesa T .
- Hrana e přitom leží v elementárním řezu oddělujícím strom T_u od zbytku grafu.
- ...

Dokončení důkazu.

- Hrana e musí být nejlehčí hrana elementárního řezu oddělujícího strom T_u od zbytku grafu, neboť případnou lehčí hranu by algoritmus potkal dříve a přidal by ji do T .
- Hrana e tedy leží v minimální kostře podle lemmatu o řezech.
- Nyní si ukážeme, že výsledný T je souvislý.
- Pokud by nebyl, uvažme dva rozdílné stromy T_u a T_v lesa T a elementární řez oddělující strom T_u od zbytku grafu.
- Protože je graf G souvislý, je tento elementární řez neprázdný a má nějakou hranu e .
- Když algoritmus zkoumal hranu e , nemohla tvořit cyklus (netvoří ho ani ve výsledném T), takže by ji algoritmus musel přidat, což je spor.
- Výstupem algoritmu je tedy kostra, která je podgrafem minimální kostry – tedy minimální kostra. □

Struktura Union-Find

Definice

Struktura Union-Find reprezentuje rozklad množiny objektů X a podporuje operace:

- **Init**(X) vytvoří strukturu, ve které je každý element z X ve vlastní množině.
- **Find**(u) vrátí identifikátor množiny obsahující element u .
- **Union**(u, v) sjednotí množiny obsahující prvky u a v (pokud jsou ze stejné množiny, struktura zůstane nezměněná).

Implementace Kruskalova algoritmu

- Kruskalův algoritmus musí nejprve seřadit **hrany**. Na to použijeme pochopitelně optimální řadící algoritmus.
- Začne s lesem izolovaných vrcholů a testuje postupně hrany, zda jejich vrcholy leží ve dvou různých stromových komponentách lesa a pokud ano, spojí tyto komponenty pomocí této hrany do větší.
- To vede na využití struktury Union-Find.
- Struktura Union-Find reprezentuje komponenty souvislosti aktuálního grafu a operace pak speciálně jsou:
 - ▶ **Init**(V) založíme strukturu – každý vrchol je v samostatné komponentě.
 - ▶ **Find**(u) vrátí identifikátor komponenty, ve které leží vrchol u .
 - ▶ Pokud pro testovanou hranu $\{u, v\}$ platí, že u a v patří do různých komponent (tj. $\text{Find}(u) \neq \text{Find}(v)$), pak **Union**(u, v) tyto komponenty spojí do jedné (přidáme $\{u, v\}$ do konstruované kostry).

Kruskalův algoritmus s Union-Find

Vstup: Souvislý graf G s vahami na hranách

Výstup: Kostra T s minimální vahou

Algoritmus **MinKostraKruskal**($G = (V, E), w: E \rightarrow \mathbb{R}$)

- (1) Uspořádej hrany podle vah: $w(e_1) \leq \dots \leq w(e_m)$
- (2) $T := (V, \emptyset)$ //pocatecni les izolovaných vrcholů
- (3) $U := \text{Init}(V)$
- (4) Pro $i := 1, \dots, m$ opakuj:
 - (5) označ u, v krajní vrcholy hrany e_i
 - (6) $a := U \rightarrow \text{Find}(u)$; $b := U \rightarrow \text{Find}(v)$;
 - (7) Pokud $a \neq b$:
 - (8) $E(T) := E(T) \cup \{e_i\}$
 - (9) $U \rightarrow \text{Union}(a, b)$
- (10) Vrať T

Časová složitost Kruskalova algoritmu

- Připomeňme, že $n = |V|$ a $m = |E|$.
- Řazení hran trvá $O(m \log m) = O(m \log n^2) = O(m \log n)$.
- Zbytek algoritmu testuje hrany pomocí operace **Find** a spojuje menší stromy do větších stromů pomocí operace **Union**.

Věta (O časové složitosti Kruskalova algoritmu)

Kruskalův algoritmus vytvoří minimální kostru v čase $O(m \log n + T_i(n) + m \cdot T_f(n) + n \cdot T_u(n))$, kde $T_i(n)$, $T_f(n)$ a $T_u(n)$ jsou časové složitosti operací **Init**, **Find** a **Union** v Union-Find struktuře nad množinou velikosti n .

Důkaz.

- Založí strukturu Union-Find.
- Při testování hran se nejvýše $2m$ -krát zavolá **Find** a $(n - 1)$ -krát se zavolá **Union**. □

Implementace struktury Union-Find pomocí pole

- Triviální přístup: použijeme pole, které každému elementu přiřadí číslo jeho množiny.
- **Find** přečte číslo množiny z pole v konstantním čase.
- **Union** při slučování množin projde všechny elementy jedné množiny a přiřadí jim číslo té druhé.

Pozorování

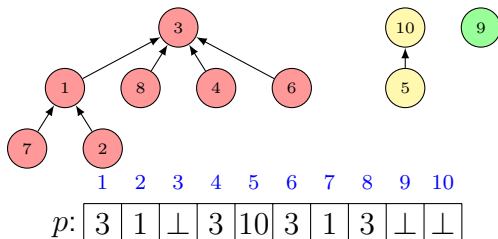
Při reprezentaci polem je časová složitost

- **Init** $T_i(n) = O(n)$,
- **Find** $T_f(n) = O(1)$ a
- **Union** $T_u(n) = O(n)$.

Časová složitost Kruskalova algoritmu by tedy byla $O(m \log n + n + m + n^2) = O(m \log n + n^2)$.

Implementace struktury Union-Find pomocí keříků

- Každou množinu, budeme reprezentovat stromem orientovaným směrem do kořene – budeme jim říkat **keříky**.
- Založená struktura je les, který má jednovrcholový strom za každý element universa.
- Vrcholy keříku odpovídají elementům příslušné množiny.
- Jako identifikátory množin použijeme element uložený v kořeni daného keříku.
- V paměti budeme keříky reprezentovat úsporně: každý element v si pamatuje „pouze“ identifikátor svého otce $p(v)$.
- Kořeny keříků v mají $p(v) = \perp$.



Find s keříky

- Operace **Find**(u) postupně vystoupá ze zadaného elementu u do kořene keříku a vrátí tento kořen.

Algoritmus **Find**(u)

- (1) Dokud $p(u) \neq \perp$:
- (2) $u := p(u)$
- (3) Vrať u

Pozorování

Časová složitost **Find**(u) je $O(\text{hloubka keříku elementu } u)$.

Union s keříky

- Do kořene v každého keříku uložíme **hloubku** keříku $H(v)$.
- Na počátku mají všechny keříky hloubku 1.
- Při slučování různě hlubokých keříků připojíme mělčí keřík pod kořen toho hlubšího a hloubka toho hlubšího se nezmění.
- Jsou-li oba keříky stejně hluboké, rozhodneme se libovolně a výsledný keřík má hloubku o jedna větší.

Algoritmus **Union**(u, v)

(1) $a := \text{Find}(u)$

(2) $b := \text{Find}(v)$

(3) Je-li $a = b$, skonči

(4) Pokud $H(a) = H(b)$:

(5) $H(a) := H(a) + 1$

//a pokračuj řádky (8)–(9)

(6) Pokud $H(a) < H(b)$:

(7) $p(a) := b$

(8) Pokud $H(a) > H(b)$:

(9) $p(b) := a$

Hloubka keříků

Lemma (o hloubce keříků)

Výše popsaný algoritmus **Union** zachovává invariant, že keřík s h hladinami obsahuje nejméně 2^{h-1} vrcholů.

Důkaz.

- Indukcí podle počtu operací **Union**.
- Na počátku algoritmu mají všechny keříky jednu hladinu a 1 vrchol (a $1 \geq 2^{1-1} = 1$).
- Nechť nyní provádíme **Union**(u, v) a počet hladin obou keříků je různý.
- Připojením mělčího keříku pod kořen toho hlubšího se počet hladin nezmění a počet vrcholů neklesne, takže nerovnost stále platí.
- ...

Hloubka keříků

Dokončení důkazu.

- Pokud mají oba keříky h hladin, platí z indukčního předpokladu, že každý z nich obsahuje minimálně 2^{h-1} vrcholů.
- Jejich sloučením tudíž vznikne keřík s $h + 1$ hladinami o alespoň $2 \cdot 2^{h-1} = 2^h$ vrcholech. □

Důsledek

Hloubka keříků během provádění výše popsaného algoritmu

Union(u, v) nepřekročí $\log n$.

Důkaz.

Strom s větším počtem hladin by podle invariantu obsahoval více než n vrcholů. □

Složitost Kruskalova algoritmu s keříky

- Z předchozího plyne, že v keříkové reprezentaci je časová složitost operace
 - ▶ **Init** $T_i(n) = O(n)$,
 - ▶ **Find** $T_f(n) = O(\log n)$ a
 - ▶ **Union** $T_u(n) = O(\log n)$ (my dokonce voláme jen na kořeny, takže lze počítat $O(1)$).
- A proto dostaneme

$$\begin{aligned} & O(m \log n + T_i(n) + m \cdot T_f(n) + n \cdot T_u(n)) = \\ & = O(m \log(n) + n + m \cdot \log(n) + n \cdot \log(n)) \in \\ & \in O(m \log n) \end{aligned}$$

Důsledek (věty o časové složitosti Kruskalova algoritmu)

Kruskalův algoritmus s keříkovou strukturou pro Union-Find vytvoří minimální kostru v čase $O(|E| \log |V|)$.

Struktura Union-Find obecně

- Kruskalův algoritmus je velmi známá aplikace datové struktury Union-Find, ale ne jediná.
- Struktura Union-Find je vhodná a přirozená pro algoritmické (snadno paralelizovatelné) řešení dalších problémů, např.
 - ▶ Konstrukce souvislých komponent grafu.
 - ▶ Detekce cyklu v grafu.
 - ▶ Konstrukce bludiště.
 - ▶ Odvozování typu proměnných v dynamických programovacích jazycích.

Co bychom po dnešku už měli znát

- Jarníkův algoritmus.
- Kruskalův algoritmus.
- Struktura Union-Find a její implementace s keříky.